

Lab 9: Scheme **lab09.zip (lab09.zip)**

Due by 11:59pm on Wednesday, November 6.

Starter Files

Download [lab09.zip \(lab09.zip\)](#).

Scheme Introduction

The 61A Scheme interpreter is included in each Scheme assignment. To start it, type `python3 scheme` in a terminal. To load a Scheme file called `f.scm`, type `python3 scheme -i f.scm`. To exit the Scheme interpreter, type `(exit)`.

Scheme Editor

All Scheme assignments include a web-based editor that makes it easy to run ok tests and visualize environments. Type `python3 editor` in a terminal, and the editor will open in a browser window (at `http://127.0.0.1:31415/`). Whatever changes you make here will also save to the original file on your computer! To stop running the editor and return to the command line, type `Ctrl-C` in the terminal where you started the editor.

The `Run` button loads the current assignment's `.scm` file and opens a Scheme interpreter, allowing you to try evaluating different Scheme expressions.

The `Test` button runs all ok tests for the assignment. Click `View Case` for a failed test, then click `Debug` to step through its evaluation.

Recommended VS Code Extensions

If you choose to use VS Code as your text editor (instead of the web-based editor), install the [vscode-scheme](https://marketplace.visualstudio.com/items?itemName=sjhuangx.vscode-scheme) (<https://marketplace.visualstudio.com/items?itemName=sjhuangx.vscode-scheme>) extension so that parentheses are highlighted.

Before:

```
1 (define foo (lambda (x y z) (if x y z)))
2
3 (foo 1 2 (print 'hi))
4
5 ((lambda (a) (print 'a)) 100)
```

After:

```
1 (define foo (lambda (x y z) (if x y z)))
2
3 (foo 1 2 (print 'hi))
4
5 ((lambda (a) (print 'a)) 100)
```

Required Questions

Getting Started Videos

Consult the drop-downs below if you need a refresher on Scheme. It's okay to skip directly to the questions and refer back here should you get stuck.

Primitive Expressions

Atomic expressions (also called *atoms*) are expressions without sub-expressions, such as numbers, boolean values, and symbols.

```
scm> 1234      ; integer
1234
scm> 123.4     ; real number
123.4
scm> #f        ; the Scheme equivalent of False in Python
#f
```

A Scheme *symbol* is equivalent to a Python name. A symbol evaluates to the value bound to that symbol in the current environment. (They are called symbols rather than names because they include `+` and other arithmetic symbols.)

```
scm> quotient   ; A symbol bound to a built-in procedure
#[quotient]
scm> +          ; A symbol bound to a built-in procedure
#[+]
```

In Scheme, *all* values except `#f` (equivalent to `False` in Python) are true values (unlike Python, which has other false values, such as `0`).

```
scm> #t
#t
scm> #f
#f
```

Call Expressions

Scheme uses Polish prefix notation, in which the operator expression comes before the operand expressions. For example, to evaluate $3 * (4 + 2)$, we write:

```
scm> (* 3 (+ 4 2))  
18
```

Just like in Python, to evaluate a call expression:

1. Evaluate the operator. It should evaluate to a procedure.
2. Evaluate the operands, left to right.
3. Apply the procedure to the evaluated operands.

Here are some examples using built-in procedures:

```
scm> (+ 1 2)  
3  
scm> (- 10 (/ 6 2))  
7  
scm> (modulo 35 4)  
3  
scm> (even? (quotient 45 2))  
#t
```

Special Forms

Define: The `define` form is used to assign values to symbols. It has the following syntax:

```
(define <symbol> <expression>)
```

```
scm> (define pi (+ 3 0.14))  
pi  
scm> pi  
3.14
```

To evaluate the `define` expression:

1. Evaluate the final sub-expression (`<expression>`), which in this case evaluates to `3.14`.
2. Bind that value to the symbol (`symbol`), which in this case is `pi`.
3. Return the symbol.

The `define` form can also define new procedures, described in the "Defining Functions" section.

If Expressions: The `if` special form evaluates one of two expressions based on a predicate.

```
(if <predicate> <if-true> <if-false>)
```

The rules for evaluating an `if` special form expression are as follows:

1. Evaluate the `<predicate>`.
2. If the `<predicate>` evaluates to a true value (anything but `#f`), evaluate and return the value of the `<if-true>` expression. Otherwise, evaluate and return the value of the `<if-false>` expression.

For example, this expression does not error and evaluates to 5, even though the sub-expression `(/ 1 (- x 3))` would error if evaluated.

```
scm> (define x 3)
x
scm> (if (> (- x 3) 0) (/ 1 (- x 3)) (+ x 2))
5
```

The `<if-false>` expression is optional.

```
scm> (if (= x 3) (print x))
3
```

Let's compare a Scheme `if` expression with a Python `if` statement:

- In Scheme:

```
(if (> x 3) 1 2)
```

- In Python:

```
if x > 3:
    1
else:
    2
```

The Scheme `if` expression evaluates to a number (either 1 or 2, depending on `x`). The Python statement does not evaluate to anything, and so the 1 and 2 cannot be used or accessed.

Another difference between the two is that it's possible to add more lines of code into the suites of the Python `if` statement, while a Scheme `if` expression expects just a single expression in each of the `<if-true>` and `<if-false>` positions.

One final difference is that in Scheme, you cannot write `elif` clauses.

Cond Expressions: The `cond` special form can include multiple predicates (like `if/elif` in Python):

```
(cond
  (<p1> <e1>)
  (<p2> <e2>)
  ...
  (<pn> <en>)
  (else <else-expression>))
```

The first expression in each clause is a predicate. The second expression in the clause is the return expression corresponding to its predicate. The `else` clause is optional; its `<else-expression>` is the return expression if none of the predicates are true.

The rules of evaluation are as follows:

1. Evaluate the predicates `<p1>`, `<p2>`, ..., `<pn>` in order until one evaluates to a true value (anything but `#f`).
2. Evaluate and return the value of the return expression corresponding to the first predicate expression with a true value.
3. If none of the predicates evaluate to true values and there is an `else` clause, evaluate and return `<else-expression>`.

For example, this `cond` expression returns the nearest multiple of 3 to `x`:

```
scm> (define x 5)
x
scm> (cond ((= (modulo x 3) 0) x)
        ((= (modulo x 3) 1) (- x 1))
        ((= (modulo x 3) 2) (+ x 1)))
6
```

Lambdas: The `lambda` special form creates a procedure.

```
(lambda (<param1> <param2> ...) <body>)
```

This expression will create and return a procedure with the given formal parameters and body, similar to a `lambda` expression in Python.

```
scm> (lambda (x y) (+ x y))          ; Returns a lambda procedure, but doesn't assign it
(lambda (x y) (+ x y))
scm> ((lambda (x y) (+ x y)) 3 4)    ; Create and call a lambda procedure in one line
7
```

Here are equivalent expressions in Python:

```
>>> lambda x, y: x + y
<function <lambda> at ...>
>>> (lambda x, y: x + y)(3, 4)
7
```

The `<body>` may contain multiple expressions. A scheme procedure returns the value of the last expression in its body.

Defining Functions

The `define` form can create a procedure and give it a name:

```
(define (<symbol> <param1> <param2> ...) <body>)
```

For example, this is how we would define the `double` procedure:

```
scm> (define (double x) (* x 2))
double
scm> (double 3)
6
```

Here's an example with three arguments:

```
scm> (define (add-then-mul x y z)
      (* (+ x y) z))
scm> (add-then-mul 3 4 5)
35
```

When a `define` expression is evaluated, the following occurs:

1. Create a procedure with the given parameters and `<body>`.
2. Bind the procedure to the `<symbol>` in the current frame.
3. Return the `<symbol>`.

The following two expressions are equivalent:

```
scm> (define add (lambda (x y) (+ x y)))
add
scm> (define (add x y) (+ x y))
add
```

Q1: Over or Under

Define a procedure `over-or-under` which takes in a number `num1` and a number `num2` and returns the following:

- -1 if `num1` is less than `num2`
- 0 if `num1` is equal to `num2`

- 1 if `num1` is greater than `num2`

NOTE. Remember that every parenthesis in Scheme makes a function call. For example, just typing `0` in the Scheme interpreter will return `0`. However, typing `(0)` will cause an Error because `0` is not a function.

Challenge: Implement this in 2 different ways using `if` and `cond`!

```
(define (over-or-under num1 num2)
  'YOUR-CODE-HERE
)
```

Use Ok to test your code:

```
python3 ok -q over_or_under
```



Q2: Make Adder

Write a procedure `make-adder` that takes a number `num` as input and returns a new procedure. This returned procedure should accept another number `inc` and return the result of `num + inc`.

Hint: To return a procedure, you can either return a `lambda` expression or define another nested procedure.

Note: `define` doesn't return the function, but `lambda` does.

Hint: Scheme will automatically return the last clause in your procedure.

You can find documentation on the syntax of `lambda` expressions in [the 61A scheme specification!](https://cs61a.org/articles/scheme-spec/#lambda) (<https://cs61a.org/articles/scheme-spec/#lambda>)

```
(define (make-adder num)
  'YOUR-CODE-HERE
)
```

Use Ok to test your code:

```
python3 ok -q make_adder
```



Q3: Compose

Write the procedure `composed`, which takes in procedures `f` and `g` and returns a new procedure. This new procedure takes in a number `x` and returns the result of calling `f` on `g` of `x`.

NOTE. Remember to use *Scheme syntax* when calling functions. The form is `(func arg)`, not `func(arg)`.

```
(define (composed f g)
  'YOUR-CODE-HERE
)
```

Use Ok to test your code:

```
python3 ok -q composed
```



Q4: Repeat

Write the procedure `repeat`, which takes in a procedure `f` and a number `n`, and outputs a new procedure. This new procedure takes in a number `x` and returns the result of calling `f` on `x` a total of `n` times. For example:

```
scm> (define (square x) (* x x))
square
scm> ((repeat square 2) 5) ; (square (square 5))
625
scm> ((repeat square 3) 3) ; (square (square (square 3)))
6561
scm> ((repeat square 1) 7) ; (square 7)
49
```

Hint: The `composed` function you wrote in the previous problem might be useful.

```
(define (repeat f n)
  'YOUR-CODE-HERE
)
```

Use Ok to test your code:

```
python3 ok -q repeat
```



Q5: Greatest Common Divisor

The Greatest Common Divisor (GCD) is the largest integer that evenly divides two positive integers.

Write the procedure `gcd`, which computes the GCD of numbers `a` and `b` using Euclid's algorithm, which recursively uses the fact that the GCD of two values is either of the following:

- the smaller value if it evenly divides the larger value, or
- the greatest common divisor of the smaller value and the remainder of the larger value divided by the smaller value

In other words, if `a` is greater than `b` and `a` is not divisible by `b`, then

```
gcd(a, b) = gcd(b, a % b)
```

You may find the provided procedures `min` and `max` helpful. You can also use the built-in `modulo` and `zero?` procedures.

```
scm> (modulo 10 4)
2
scm> (zero? (- 3 3))
#t
scm> (zero? 3)
#f
```

```
(define (max a b) (if (> a b) a b))
(define (min a b) (if (> a b) b a))
(define (gcd a b)
  'YOUR-CODE-HERE
)
```

Use Ok to test your code:

```
python3 ok -q gcd
```



Check Your Score Locally

You can locally check your score on each question of this assignment by running

```
python3 ok --score
```

This does NOT submit the assignment! When you are satisfied with your score, submit the assignment to Gradescope to receive credit for it.

Submit Assignment

If you are in a regular section of CS 61A, fill out this lab attendance and feedback form (<https://forms.gle/dHxj8gttNWRy6Ptm9>). (If you are in the mega section, you don't need to fill out the form.)

Then, submit this assignment by uploading any files you've edited **to the appropriate Gradescope assignment**. Lab 00 ([../lab00/#submit-with-gradescope](https://gradescope.com/submissions/lab00/#submit-with-gradescope)) has detailed instructions.